

Jumble — Database Schema

Author: Nusrath Fathima **Last updated:** 17 September 2026 **Database:** PostgreSQL 16

Companion to: Jumble — Project Spec (step 1 of 3)

0. Decisions carried in from the spec

The three open questions from section 11 of the spec are resolved here as follows. Each is a real tradeoff, so each is worth being able to defend.

Tag weights live in their own table (`child_tag_weight`) rather than being recomputed from completion history on every request. The suggestions endpoint is the hot path — it runs every time a parent opens the app — so reads there should be cheap. The cost is that every feedback event must maintain the derived rows. This is a deliberate read-performance-over-write-simplicity trade, and it is the single most interesting thing in this schema to talk about in an interview.

The repeat-suppression window is configurable, defaulting to 14 days. It is passed into the query as a parameter rather than hard-coded, so it can be tuned without a migration once the activity library's real size is known.

One parent owns each child. Household sharing stays a stretch feature. Adding it later means introducing a `household` table and repointing one foreign key — an additive migration, not a destructive one.

1. Entity overview

Table	Role
parent	Account, credentials, location for weather lookups
child	A child belonging to a parent
tag	Controlled vocabulary of activity categories
child_tag	A child's declared starting interests
child_tag_weight	Learned per-child affinity score per tag — the ranking input
activity	A library item
activity_tag	Which tags an activity belongs to
material	A supply item that activities may require
activity_material	What an activity needs, and whether it's optional
parent_inventory	What a household has on hand
completion	A child did an activity, with a rating
weather_cache	Cached daily forecast per location

2. Design decisions worth explaining

Age is derived, never stored. `child.birth_date` is stored and age is computed at query time. Storing an integer age would silently rot — a child recorded as 5 stays 5 forever, and the age-range filter would slowly start recommending the wrong things. The app collects only month and year, storing day as `01`, so no child's exact date of birth is held. That is both a privacy decision and a correctness decision.

Tags are a lookup table, not free text. A `tag` table with a foreign key means the weights table can join cleanly, tag names can be renamed in one place, and typos can't fragment the taxonomy.

Duration is total time, cleanup included. `activity.duration_minutes` is documented as realistic end-to-end time. Activity blogs systematically understate this by ignoring setup and tidying, and the whole product promise depends on a parent trusting that "30 minutes" means 30 minutes.

Required versus optional materials. `activity_material.is_optional` matters because the availability filter only enforces non-optional rows. Without this flag, an activity needing glitter as a nice-to-have would be hidden from every household without glitter.

Inventory belongs to the parent, not the child. Supplies are a household resource. Modelling it per-child would duplicate rows and create contradictions between siblings.

Enums as `VARCHAR` with a `CHECK` constraint rather than native Postgres enum types. Native enums are awkward to evolve and awkward for Hibernate to map; a checked varchar maps cleanly to `@Enumerated(EnumType.STRING)` and altering the allowed set is a simple constraint change.

Activities are retired, not deleted. `activity.is_active` preserves referential integrity with historical `completion` rows. A parent's history shouldn't break because a library item was pulled.

Foreign keys are explicitly indexed. Postgres does not create indexes on foreign keys automatically, and every one of these is used in a join on the suggestions path.

3. DDL

```
-- -----  
-- Accounts  
-- -----  
  
CREATE TABLE parent (  
    id            BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    email         VARCHAR(255) NOT NULL UNIQUE,  
    password_hash VARCHAR(100) NOT NULL,  
    display_name  VARCHAR(100),  
    -- Coarse location for weather; NULL disables weather filtering  
    latitude      NUMERIC(8,5),  
    longitude     NUMERIC(8,5),  
    created_at    TIMESTAMPTZ NOT NULL DEFAULT now(),  
    updated_at    TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
  
CREATE TABLE child (  
    id            BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    parent_id    BIGINT NOT NULL REFERENCES parent(id) ON DELETE CASCADE,  
    name         VARCHAR(80) NOT NULL,  
    -- Month/year only; day is always stored as 01  
    birth_date   DATE NOT NULL,  
    created_at   TIMESTAMPTZ NOT NULL DEFAULT now(),  
    updated_at   TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
  
CREATE INDEX idx_child_parent ON child(parent_id);  
  
-- -----  
-- Taxonomy  
-- -----  
  
CREATE TABLE tag (  
    id            SMALLINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    slug         VARCHAR(40) NOT NULL UNIQUE,  
    display_name  VARCHAR(60) NOT NULL  
);  
  
-- A child's declared starting interests, captured at onboarding.  
-- Used to seed child_tag_weight; feedback takes over from there.  
CREATE TABLE child_tag (  
    child_id BIGINT NOT NULL REFERENCES child(id) ON DELETE CASCADE,  
    tag_id   SMALLINT NOT NULL REFERENCES tag(id) ON DELETE CASCADE,
```

```

    PRIMARY KEY (child_id, tag_id)
);

-- Learned affinity. One row per (child, tag), seeded at child creation.
CREATE TABLE child_tag_weight (
    child_id    BIGINT    NOT NULL REFERENCES child(id) ON DELETE CASCADE,
    tag_id      SMALLINT NOT NULL REFERENCES tag(id)    ON DELETE CASCADE,
    weight      REAL      NOT NULL DEFAULT 1.0,
    updated_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (child_id, tag_id),
    CONSTRAINT chk_weight_range CHECK (weight >= 0.2 AND weight <= 3.0)
);

-- -----
-- Activity library
-- -----

CREATE TABLE activity (
    id                BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    slug              VARCHAR(80) NOT NULL UNIQUE,
    title             VARCHAR(140) NOT NULL,
    summary           VARCHAR(300) NOT NULL,
    instructions      TEXT          NOT NULL,
    min_age_years     SMALLINT      NOT NULL,
    max_age_years     SMALLINT      NOT NULL,
    -- Realistic total time INCLUDING setup and cleanup
    duration_minutes  SMALLINT      NOT NULL,
    mess_level        SMALLINT      NOT NULL,    -- 1 none, 2 some, 3 significant
    location_type     VARCHAR(10) NOT NULL,    -- INDOOR | OUTDOOR | EITHER
    needs_adult       BOOLEAN       NOT NULL DEFAULT false,
    is_active         BOOLEAN       NOT NULL DEFAULT true,
    created_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT chk_age_range    CHECK (min_age_years <= max_age_years),
    CONSTRAINT chk_mess_level   CHECK (mess_level BETWEEN 1 AND 3),
    CONSTRAINT chk_location     CHECK (location_type IN ('INDOOR', 'OUTDOOR', 'EITHER')),
    CONSTRAINT chk_duration     CHECK (duration_minutes > 0)
);

CREATE INDEX idx_activity_active ON activity(is_active)
    WHERE is_active;

CREATE TABLE activity_tag (
    activity_id BIGINT    NOT NULL REFERENCES activity(id) ON DELETE CASCADE,
    tag_id      SMALLINT NOT NULL REFERENCES tag(id)      ON DELETE CASCADE,

```

```

        PRIMARY KEY (activity_id, tag_id)
    );

CREATE INDEX idx_activity_tag_tag ON activity_tag(tag_id);

-- -----
-- Materials and household inventory
-- -----

CREATE TABLE material (
    id            SMALLINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    slug          VARCHAR(50) NOT NULL UNIQUE,
    display_name  VARCHAR(80) NOT NULL,
    category      VARCHAR(30) NOT NULL, -- KITCHEN | CRAFT | RECYCLING | OUTDOOR | OTHER
    -- Pre-ticked during onboarding so parents aren't faced with 80 checkboxes
    is_common     BOOLEAN      NOT NULL DEFAULT false,
    CONSTRAINT chk_material_category
        CHECK (category IN ('KITCHEN','CRAFT','RECYCLING','OUTDOOR','OTHER'))
);

CREATE TABLE activity_material (
    activity_id BIGINT NOT NULL REFERENCES activity(id) ON DELETE CASCADE,
    material_id SMALLINT NOT NULL REFERENCES material(id) ON DELETE RESTRICT,
    -- Optional materials do NOT block a suggestion
    is_optional BOOLEAN NOT NULL DEFAULT false,
    PRIMARY KEY (activity_id, material_id)
);

CREATE INDEX idx_activity_material_material ON activity_material(material_id);

CREATE TABLE parent_inventory (
    parent_id BIGINT NOT NULL REFERENCES parent(id) ON DELETE CASCADE,
    material_id SMALLINT NOT NULL REFERENCES material(id) ON DELETE CASCADE,
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (parent_id, material_id)
);

-- -----
-- History and feedback
-- -----

CREATE TABLE completion (
    id            BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    child_id      BIGINT NOT NULL REFERENCES child(id) ON DELETE CASCADE,
    activity_id   BIGINT NOT NULL REFERENCES activity(id) ON DELETE RESTRICT,

```

```

    completed_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    rating        VARCHAR(10) NOT NULL,    -- LOVED | OK | SKIPPED
    note          VARCHAR(500),
    CONSTRAINT chk_rating CHECK (rating IN ('LOVED','OK','SKIPPED'))
);

-- Serves the repeat-suppression subquery and the history screen
CREATE INDEX idx_completion_child_recent
    ON completion(child_id, completed_at DESC);
CREATE INDEX idx_completion_child_activity
    ON completion(child_id, activity_id);

-- -----
-- Weather cache
-- -----
-- One row per coarse location per day. location_key is lat/lon rounded
-- to 1 decimal place (~11km), which collapses a whole neighbourhood into
-- a single upstream API call.
CREATE TABLE weather_cache (
    id                BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    location_key       VARCHAR(20) NOT NULL,
    forecast_date      DATE        NOT NULL,
    outdoor_friendly   BOOLEAN     NOT NULL,
    condition           VARCHAR(40),
    temp_c             NUMERIC(4,1),
    precipitation_chance SMALLINT,
    fetched_at         TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT uq_weather UNIQUE (location_key, forecast_date)
);

```

4. Seeding the weights

When a child is created, one `child_tag_weight` row is inserted per tag in the taxonomy. Tags the parent selected as interests start at `1.5`; everything else starts at `1.0`. This means a brand-new child gets sensible suggestions immediately rather than a cold-start problem, and the declared interests fade in influence naturally as real feedback accumulates.

```
INSERT INTO child_tag_weight (child_id, tag_id, weight)
SELECT :childId,
       t.id,
       CASE WHEN ct.tag_id IS NOT NULL THEN 1.5 ELSE 1.0 END
FROM tag t
LEFT JOIN child_tag ct
  ON ct.tag_id = t.id AND ct.child_id = :childId;
```

5. Updating weights on feedback

When a completion is recorded, the weights for that activity's tags move:

Rating	Adjustment
LOVED	+0.30
OK	+0.05
SKIPPED	-0.20

```
UPDATE child_tag_weight w
SET weight      = LEAST(3.0, GREATEST(0.2, w.weight + :delta)),
    updated_at = now()
WHERE w.child_id = :childId
    AND w.tag_id IN (SELECT tag_id FROM activity_tag WHERE activity_id = :activityId);
```

The `LEAST / GREATEST` clamp mirrors the `chk_weight_range` constraint and matters more than it looks: without it, a child who loves building activities would eventually have a building weight so high that nothing else could ever surface. The clamp keeps the system responsive to changing tastes.

6. The suggestions query

This is the centrepiece — everything above exists to make this one query work.

```

WITH ctx AS (
    SELECT c.id          AS child_id,
           c.parent_id,
           EXTRACT(YEAR FROM age(current_date, c.birth_date))::int AS age_years
    FROM child c
    WHERE c.id = :childId
)
SELECT a.id,
       a.title,
       a.duration_minutes,
       COALESCE(SUM(w.weight), 0) AS score
FROM activity a
CROSS JOIN ctx
LEFT JOIN activity_tag at
    ON at.activity_id = a.id
LEFT JOIN child_tag_weight w
    ON w.tag_id = at.tag_id AND w.child_id = ctx.child_id
WHERE a.is_active
    AND ctx.age_years BETWEEN a.min_age_years AND a.max_age_years
    AND a.duration_minutes <= :availableMinutes
    AND a.mess_level <= :maxMessLevel
    -- 'EITHER' from the caller means "no preference", so it filters nothing
    AND (:locationType = 'EITHER'
        OR a.location_type = 'EITHER'
        OR a.location_type = :locationType)
    -- not done too recently
    AND NOT EXISTS (
        SELECT 1 FROM completion cp
        WHERE cp.child_id = ctx.child_id
            AND cp.activity_id = a.id
            AND cp.completed_at > now() - make_interval(days => :repeatWindowDays)
    )
    -- every non-optional material is in the household inventory
    AND NOT EXISTS (
        SELECT 1
        FROM activity_material am
        WHERE am.activity_id = a.id
            AND NOT am.is_optional
            AND NOT EXISTS (
                SELECT 1 FROM parent_inventory pi
                WHERE pi.parent_id = ctx.parent_id
                    AND pi.material_id = am.material_id
            )
    )

```

```
)  
GROUP BY a.id, a.title, a.duration_minutes  
ORDER BY score DESC, random()  
LIMIT 5;
```

Two things here are worth flagging as interview material.

The nested `NOT EXISTS` on materials is relational division — "find activities for which there does not exist a required material that the household does not have." Relational division is a genuinely awkward thing to express in SQL and most developers reach for a `COUNT(*) = COUNT(*)` join instead. Being able to explain why the double-negative formulation is both correct and index-friendly is a strong signal.

`ORDER BY score DESC, random()` breaks ties randomly instead of falling back to insertion order. Without it, two activities with identical scores would surface in the same order every single time, and the app would feel stale even though the ranking is technically correct. A one-word fix to a product problem.

7. Feeding the "explain this suggestion" feature

The explanation is assembled at request time from facts already in hand, so nothing extra needs storing:

- the child's top-weighted tags that this activity matches ("Emma has loved building activities")
- the constraints that were applied ("you have 30 minutes", "low mess")
- material availability ("you have everything for this")
- weather, where it influenced the result ("it's dry out")

Because the scoring is transparent weighted matching, the explanation is a faithful description of the computation rather than a plausible story told after the fact. That distinction is the whole argument for choosing this approach over a black-box model.

8. Validation

Every statement in this document was executed against a real PostgreSQL 16 instance before the document was issued. The DDL creates cleanly, and the following behaviours were verified with seeded fixture data (one parent, one six-year-old, six activities, seven materials, a household stocked with everything except glitter and chalk):

#	Check	Result
1	Full DDL applies without error	Pass
2	Weight seeding — declared interests at 1.5, all others at 1.0	Pass
3	Age derived correctly from <code>birth_date</code>	Pass
4	Age range excludes a toddler activity from a six-year-old	Pass
5	Missing required material hides an activity	Pass
6	Missing optional material does <i>not</i> hide an activity	Pass
7	Time budget filter drops activities that don't fit	Pass
8	Mess-level filter drops messy activities	Pass
9	Location request returns matching plus <code>EITHER</code> activities	Pass
10	Repeat suppression hides an activity after completion	Pass
11	<code>LOVED</code> feedback raises the right tag weights, and re-ranks	Pass
12	Upper clamp holds at 3.0 under repeated positive feedback	Pass
13	Lower clamp holds at 0.2 under repeated negative feedback	Pass
14	<code>chk_weight_range</code> rejects an out-of-range write	Pass
15	Deleting a parent cascades to children, weights and completions while leaving the activity library intact	Pass

One bug was found and fixed during this validation, and it's worth knowing about because it is the kind of thing that survives code review. The original location filter read `(a.location_type = 'EITHER' OR a.location_type = :locationType)`. That works when a parent asks specifically for indoor or outdoor, but when they express *no* preference — passing `EITHER` — it silently excluded every activity tagged strictly `INDOOR` or `OUTDOOR`, leaving only the flexible ones. A parent with no constraints would have received the fewest suggestions, which is precisely backwards. The corrected clause treats a caller-supplied `EITHER` as "filter nothing."

9. Open items for step 3 (scaffolding)

1. **Migration tool** — Flyway or Liquibase. Flyway's plain-SQL migrations suit a schema like this one; the DDL above becomes `v1__initial_schema.sql` more or less verbatim.

2. **Auth mechanism** — session cookies or JWT. Session cookies are simpler and safer by default for a browser-only client; JWT is the more common interview talking point. Worth a deliberate choice rather than a default.
3. **Seed data volume** — the library needs roughly 60–80 activities and 40–50 materials before recommendations feel non-repetitive. Worth planning how that content gets written.
4. **ON DELETE RESTRICT on completion.activity_id** means a library activity can never be hard-deleted once anyone has done it. That is intentional, and **is_active** is the escape hatch — confirm you're happy with it.